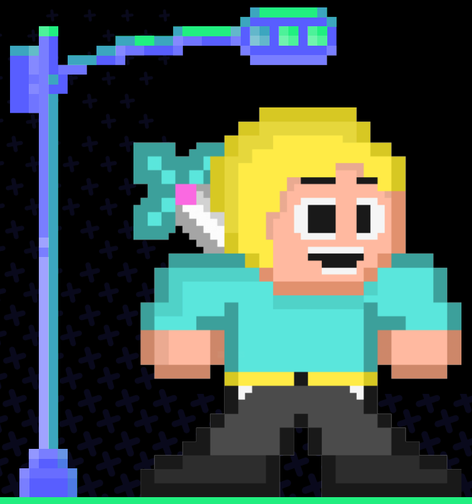
UTN-FRC
INGENIERÍA Y CALIDAD DE SOFTWARE

PUNTUACIÓN MÁS ALTA 2500

JUGADOR 1

DESARROLLO DIRIGIDO POR PRUEBAS (TDD)



- NO PRUEBES EL CÓDIGO. DISEÑALO CON PRUEBAS.

START

MENU 01 07 12

TDD

TEMARIO

DESARROLLO TRADCIONAL

TDD - CONCEPTOS

CICLO DEL TDD

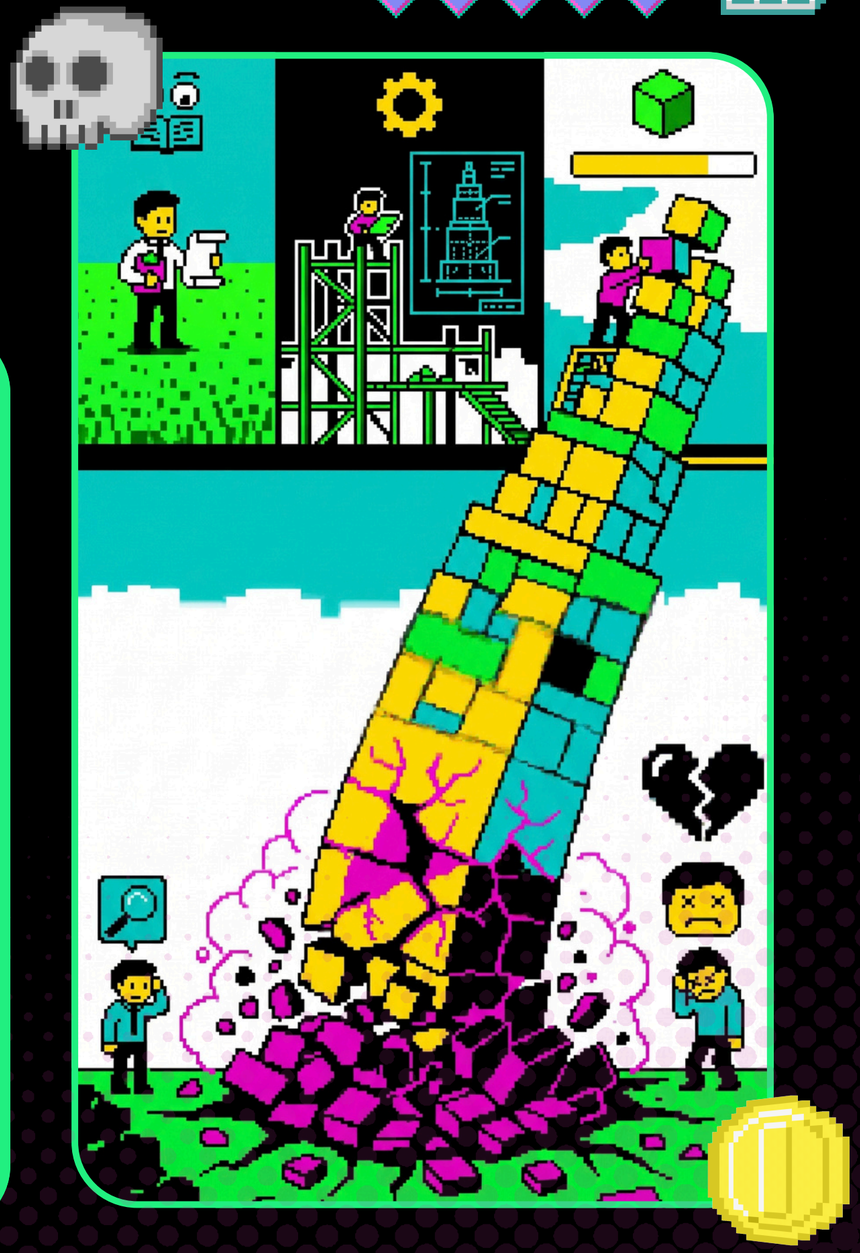
PATRONES

TAXI MOBILE

MENU 01 07 12

ENFOQUE TRADICIONAL

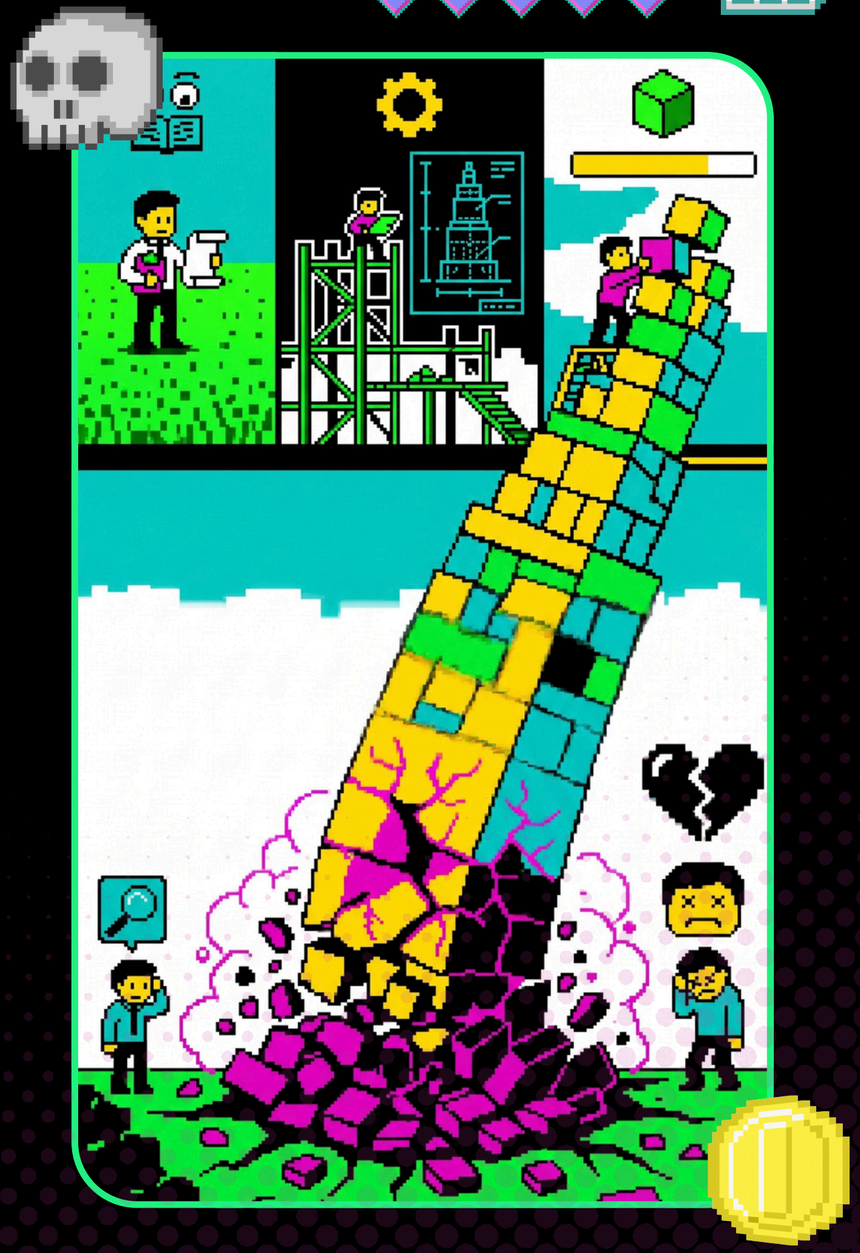
1. NEGOCIO
2. REQUERIMIENTOS
3. ANÁLISIS.
4. DISEÑO.
5. IMPLEMENTACIÓN.
6. PRUEBA



MENU 01 07 12

EL PROBLEMA DE PROBAR AL FINAL

- ACOPLAMIENTO INCIDENTAL
- DEUDA TÉCNICA Y MIEDO AL CAMBIO
- COSTO ASINTÓTICO DEL ERROR



SIGN IN

TDD

TDD

NO ES TESTING, ES DESARROLLO

KENT BECK

MENU

TDD

ES UNA ACTIVIDAD DE ESPECIFICACIÓN Y DISEÑO.

- EL ¿QUÉ?

Denifimos ***qué*** debe hacer el software

- ANTES DE DECIDIR EL CÓMO

Pasamos a un enfoque PROACTIVO

- CAMBIO DE MENTALIDAD

De escribir código y cruzar los dedos

a

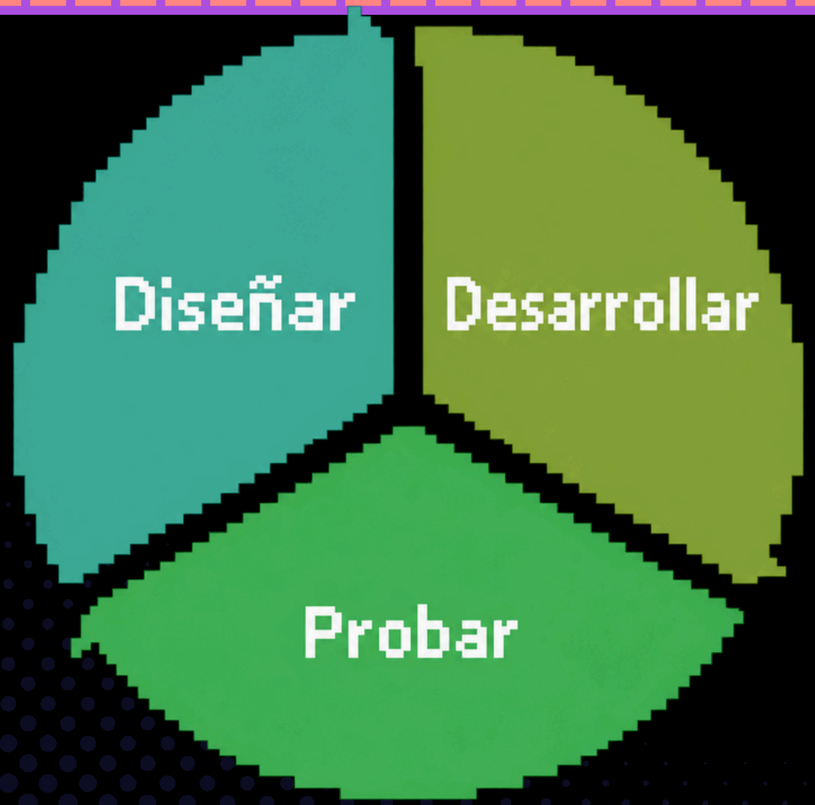
Definir el éxito y construir hacia él

MENU

TDD

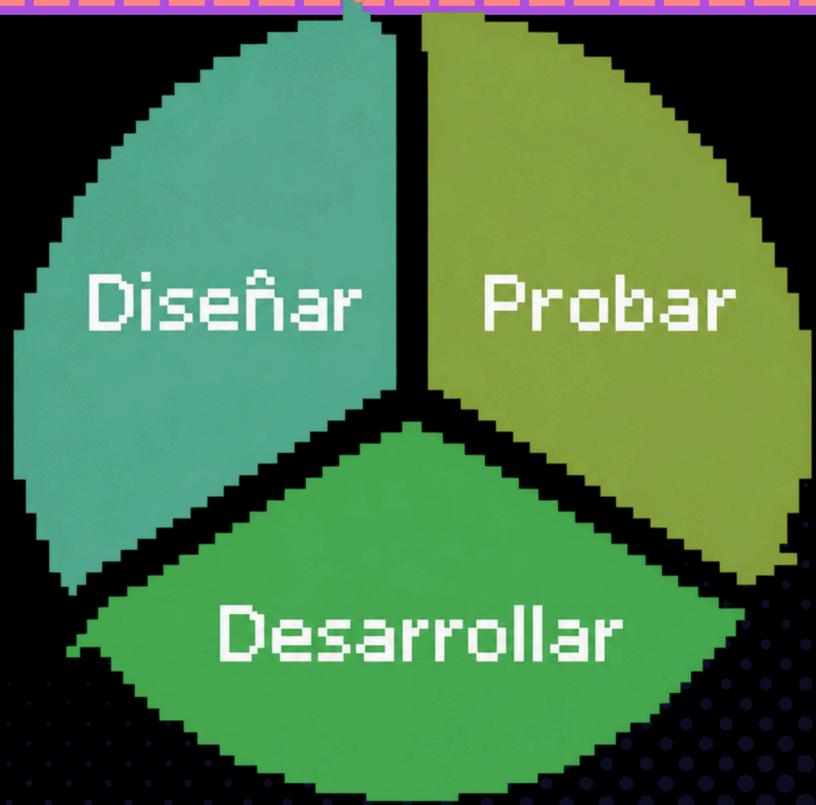
TÉCNICA DE DISEÑO DE SOFTWARE

Desarrollo tradicional



ES UN ENFOQUE REACTIVO.

TDD



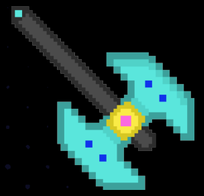
ES UN ENFOQUE PROACTIVO.

VOLVER AL PROGRAMA

01 07 12

LAS 3 LEYES DE TDD

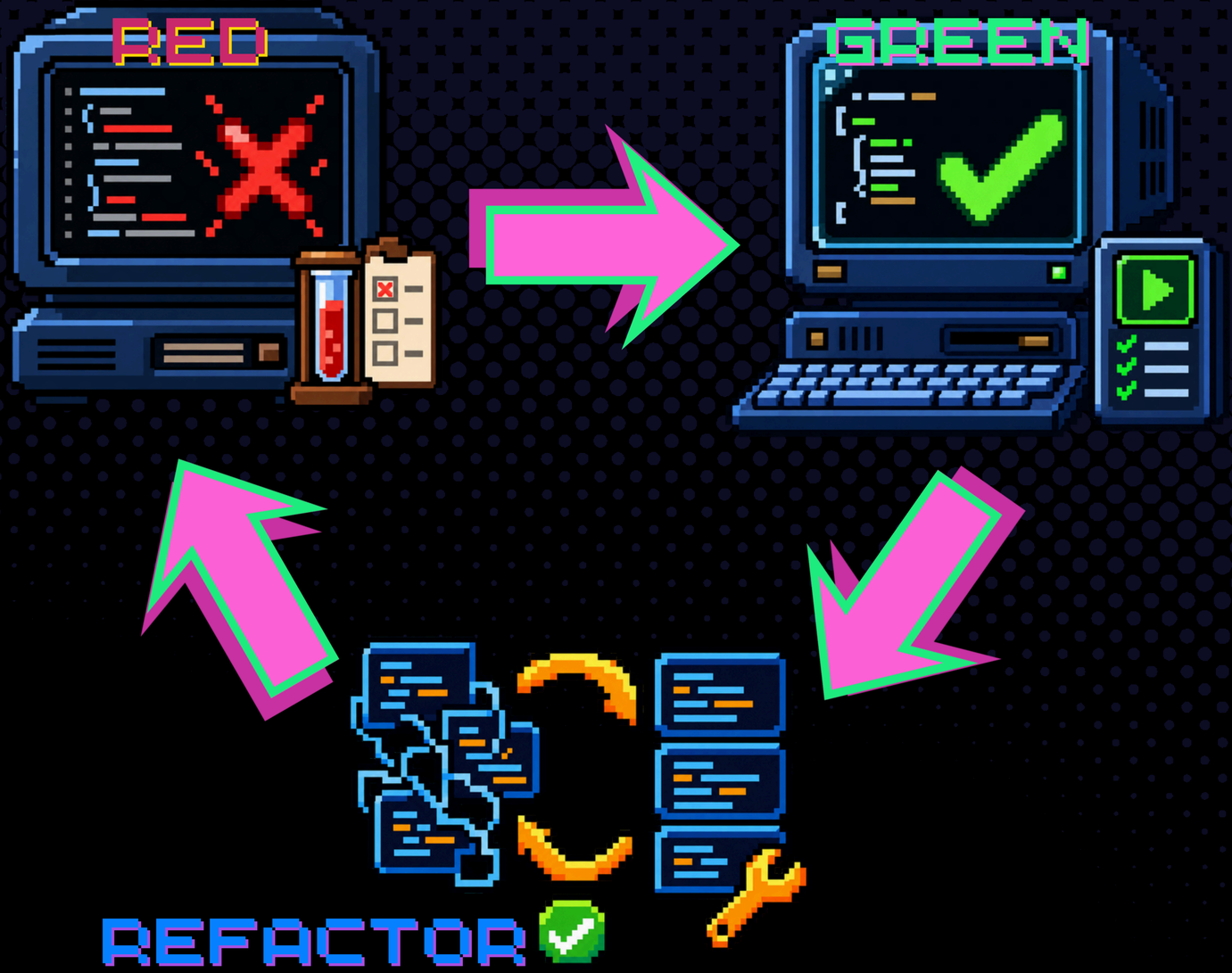
"NO ESCRIBIRÁS UNA LÍNEA DE CÓDIGO DE PRODUCCIÓN SIN ANTES HABER ESCRITO UN TEST UNITARIO QUE FALLE"



"NO ESCRIBAS MÁS CÓDIGO DE PRUEBA DEL NECESARIO PARA PROVOCAR UN FALLO. SI EL CÓDIGO NI SIQUIERA COMPILA, ESO YA CUENTA COMO FALLO."

"NO ESCRIBIRÁS MÁS CÓDIGO DE PRODUCCIÓN QUE EL MÍNIMO INDISPENSABLE PARA QUE EL TEST PASE."

JUGADOR 1



EL ALMA DE TDD - EL CICLO RED-GREEN-REFACTOR

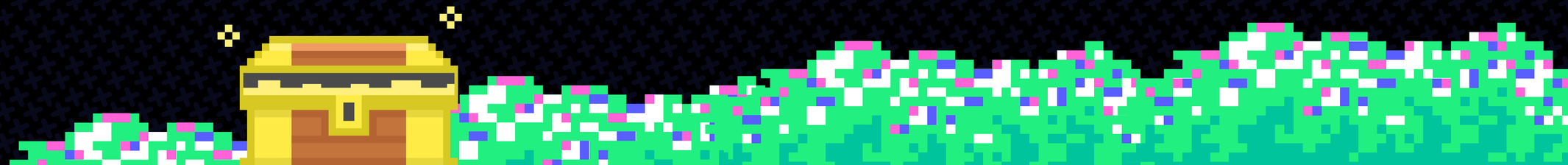
RED

Trabajamos de manera declarativa

DECLARAMOS LA INTENCIÓN:
“QUIERO QUE AL PASARLE **ESTOS** DATOS, EL SISTEMA ME DEVUELVA **ESTE** RESULTADO”

```
def test_sumar_dos_numeros_EXITO():
    //Precondiciones
    calculadoraDePrueba = new Calculadora()
    //Pasos
    resultado = calculadoraDePrueba.sumar(a = 1,b = 2)
    //Resultado
    assert(resultado == 3)

Class Calculadora:
    def sumar(a,b):
            return “No implementado” //Puede ser pass directamente
```

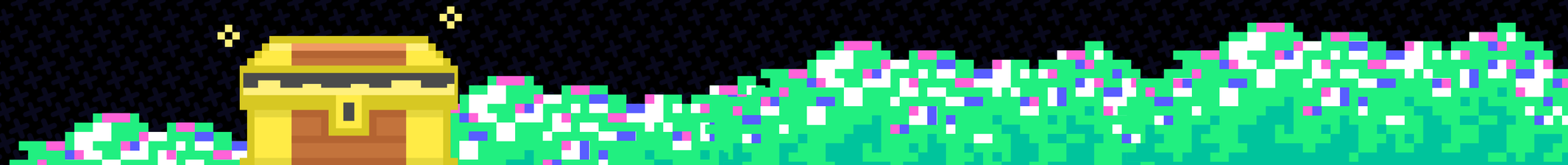


GREEN

PASAMOS A UN ENFOQUE IMPERATIVO (CÓMO).
ESCRIBIMOS EL CÓDIGO **MÍNIMO** PARA QUE LA PRUEBA PASE.

```
def test_sumar_dos_numeros_EXITO():
    //Precondiciones
    calculadoraDePrueba = new Calculadora()
    //Pasos
    resultado = calculadoraDePrueba.sumar(a = 1,b = 2)
    //Resultados
    assert(resultado == 3)

Class Calculadora:
    def sumar(a,b):
        return 3
```

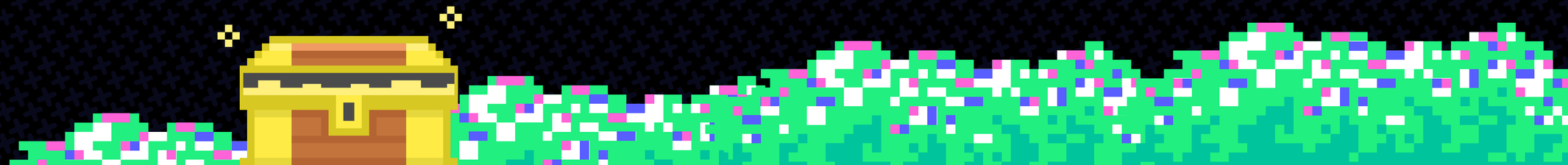


REFACTOR

REFACTORIZAR ES MODIFICAR LA ESTRUCTURA INTERNA DEL SOFTWARE **SIN ALTERAR SU COMPORTAMIENTO EXTERNO**.

```
def test_sumar_dos_numeros_EXITO():
    //Precondiciones
    calculadoraDePrueba = new Calculadora()
    //Pasos
    resultado = calculadoraDePrueba.sumar(a = 1,b = 2)
    //Resultados
    assert(resultado == 3)

Class Calculadora:
   def sumar(a: Number,b: Number) -> Number:
       return 3
```

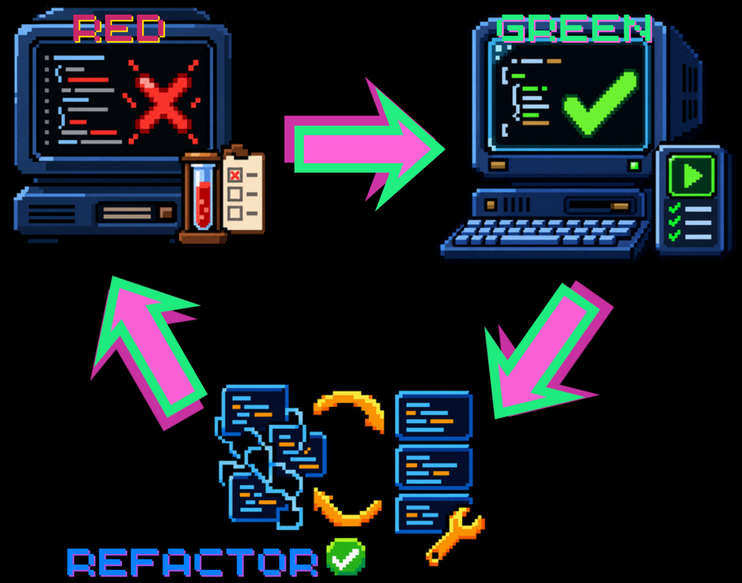


COMIENZA DE NUEVO EL CICLO

RED → GREEN → REFACTOR

```
def test_sumar_dos_numeros_EXITO():
     //Precondiciones
    calculadoraDePrueba = new Calculadora()
    //Pasos
    resultado = calculadoraDePrueba.sumar(a = 3,b = 2)
    //Resultados
    assert(resultado == 5) //nueva comparación a + b

Class Calculadora:
   def sumar(a: Number,b: Number) → Number:
       return a + b
```



PUNTUACIÓN MÁS ALTA 2500

JUGADOR 2

PATRONES Y BUENAS PRACTICAS

ESTRATEGIAS, DECISIONES DE DISEÑO, ATAJOS MENTALES O HASTA TRUCOS QUE UTILIZA UN PROGRAMADXR PARA TRANSICIONAR DE UNA FASE A OTRA DENTRO DEL CICLO.

TDD

RED

- STARTER TEST
- ONE STEP TEST
- EXPLANATION TEST
- LEARNING TEST
- ANOTHER TEST
- REGRESSION TEST

GREEN

- FAKE IT('TIL YOU MAKE IT)
- TRIANGULACIÓN
- OBVIOUS IMPLEMENTATION

ONE TO MANY

DISEÑO DE TEST

- TEST LIST
- ISOLATED TEST
- TEST FIRST
- ASSERT TEST
- TEST DATA
- EVIDENT DATA

DISEÑO DE TEST

- PATRONES DE TESTING
- PATRONES DE DISEÑO
- XUNIT PATTERNS
  - ASSERTION
  - FIXTURE
  - EXTERNAL FIXTURE
  - TEST METHOD
  - EXCEPTION TEST
  - ALLTEST

CASO TAXI MOBILE

Ver mapa de taxis 5

Como Administrador de la Central quiero ver la ubicación de todos los taxis de la central y si tienen viajes en curso para saber la disponibilidad actual

Criterios de Aceptación:

- Se muestran taxis en todos los estados (Verde: Libre, Amarillo: Solicitado, Rojo: Ocupado, Negro: Fuera de Servicio).
- Los taxis que se mostrarán deben tener habilitado el sistema de ubicación.
- Pasando el mouse sobre el ícono del auto deben poder visualizarse los datos del viaje si el taxi está ocupado (hora de inicio y costo) y los datos del pasajero (nombre, apellido y número de teléfono celular).
- Si el taxi está solicitado, sólo se visualizarán los datos del pasajero.
- Podrán utilizarse filtros por Barrio, por Estado y por chapa del taxi

Pruebas de Usuario

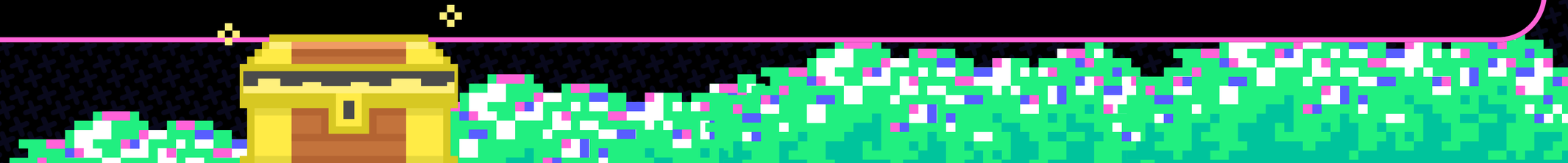
- Probar visualizar un conjunto de taxis libres (pasa)
- Probar visualizar un conjunto de taxis solicitados (pasa)
- Probar visualizar un conjunto de taxis fuera de servicio (pasa)
- Probar visualizar un conjunto de taxis ocupados (pasa)
- Probar visualizar un conjunto de taxis libres, solicitados, fuera de servicio y ocupados (pasa)
- Probar visualizar un taxi conectado al sistema por número de chapa (pasa)
- Probar visualizar en el mapa un taxi no conectado al sistema de ubicación (falla)
- Probar visualizar un conjunto de taxis en un Barrio inexistente (falla)

RED

```
def test_mostrar_mapa_taxis_libres_en_barrio_exito ():
     //Precondiciones
    usuarios = {[{user: “salvita”, rol: “Administrador de la Central}]}
    loguear(salvita)
    configurar_ciudad_prueba(“Córdoba Capital”)
    taxis = [{'chapa': 'ABC123', 'estado': 'Libre', 'barrio': 'General Paz', 'ubicacion': (10, 20)}]

   //Pasos
   mapa_mostrado = mostrar_mapa(filtros = [“barrio”, “estado”],  barrio = “General Paz”, chapa = “”, estado = “Libre”)
   //Resultado
     assert (mapa_mostrado.ciudad.barrio == “General Paz”, “No existe tal barrio para esa ciudad ingresada”)
    assert all(taxi.barrio == “General Paz” for taxi in mapa_mostrado.taxis, “No todos los taxis están dentro del area filtrada ”)
     assert all(taxi.estado.nombre == “Libre” for taxi in mapa_mostrado.taxis, “No todos los taxis están libres”)
     assert all(taxi.estado.icono == “verde” for taxi in mapa_mostrado.taxis, “No todos los taxis tienen el icono verde”)

def mostrar_mapa(filtros, barrio, chapa, estado):
     pass
```



GREEN

```
def test_mostrar_mapa_taxis_libres_en_barrio_exito ():
    //Precondiciones
    usuarios = {[{user: “salvita”, rol: “Administrador de la Central}]}
    loguear(salvita)
    configurar_ciudad_prueba(“Córdoba Capital”)
   //cargo los datos al sistema.
    taxis = [{'chapa': 'ABC123', 'estado': 'Libre', 'barrio': 'General Paz', 'ubicacion': (10, 20)}]

   //Pasos
   mapa_mostrado = mostrar_mapa(filtros = [“barrio”, “estado”],  barrio = “General Paz”, chapa = “”, estado = “Libre”)
   //Resultado
    assert(mapa_mostrado.ciudad.barrio == “General Paz”, “No existe tal barrio para esa ciudad ingresada”)
     assert all(taxi.barrio == “General Paz” for taxi in mapa_mostrado.taxis, “No todos los taxis están dentro del area filtrada ”)
    assert all(taxi.estado.nombre == “Libre” for taxi in mapa_mostrado.taxis, “No todos los taxis están libres”)
    assert all(taxi.estado.icono == “verde” for taxi in mapa_mostrado.taxis, “No todos los taxis tienen el icono verde”)

def mostrar_mapa(filtros, barrio, chapa, estado):
    return new Mapa([taxi={chapa = "ABC123", estado = {nombre = “Libre”, icono = “verde”}, barrio = “General Paz”})
```

REFACTOR

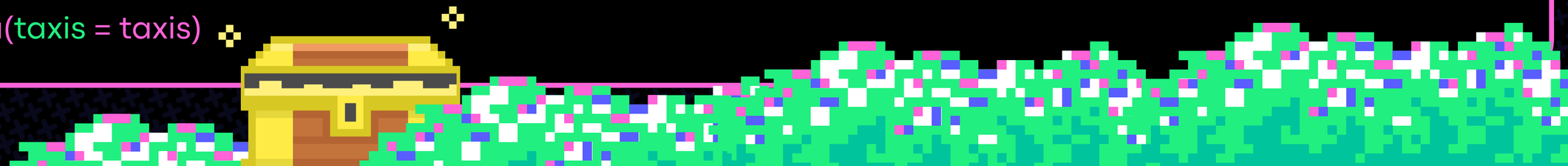
```
def mostrar_mapa(filtros: String[], barrio: Barrio, chapa: String, estado: Estado) -> Mapa:
    estado_libre = new Estado(
        nombre = "Libre",
        icono = "verde"
    )

    barrio_general_paz = new Barrio(
        nombre = "General Paz"
    )

    taxi = new Taxi(
        chapa = "ABC123",
        estado = estado_libre,
        barrio = barrio_general_paz
    )

    taxis = [taxi]

    return new Mapa(taxis = taxis)
```



¿CÓMO SIGUE?

1. Volver a inciar el ciclo.
2. Considerar el resto de casos de pruebas, añadiendo las alternativas planteadas en la user.
3. Refactorizar.

Hacer TDD sin refactorizar es simplemente escribir código malo con pruebas.

EJEMPLO FINAL

```
def mostrar_mapa(filtros: Filtros, barrio: Barrio, chapa: String, estado: Estado) -> Mapa:
    // Caso de prueba taxis desconectados
    if (filtros== ‘Mostrar todos los taxis’)
        taxis = buscarTaxiEnBarrio(barrio)
        (ubicación == None) //ubicación desconectada
        taxis = buscarTaxiDesconectado (ubicacion)

    // Caso de prueba mostrar taxis en distintos estados en un barrio
    if (filtros== ‘Mostrar taxis por estado y por barrio’)
      taxis = buscarTaxiPorEstadoYPorBarrio (barrio, estado)

    // Caso de prueba mostrar taxis por chapa
    if (filtros == ‘Mostrar taxis por chapa’)
      taxis = buscarTaxiPorChapa (chapa)

    // Caso de prueba mostrar taxis por barrio y datos de viaje
    if (filtros== ‘Mostrar taxis por barrio’)
      taxisFiltrados = buscarTaxiEnBarrio (barrio)
     //foreach simplificado
      if (taxis[i].getEstado() == ‘Solicitado’ && filtros== ‘Mostrar datos viaje’)
        taxis.append(
          taxis[i].getChapa()
          taxis[i].getViajeActual().getPasajero().getNombre()
          taxis[i].getViajeActual().getPasajero().getTelefono()
          taxis[i].getViajeActual().getFechaYHora()
        )
return new Mapa(taxis=taxis)
```

Pruebas de Usuario

- Probar visualizar un conjunto de taxis libres (pasa)
- Probar visualizar un conjunto de taxis solicitados (pasa)
- Probar visualizar un conjunto de taxis fuera de servicio (pasa)
- Probar visualizar un conjunto de taxis ocupados (pasa)
- Probar visualizar un conjunto de taxis libres, solicitados, fuera de servicio y ocupados (pasa)
- Probar visualizar un taxi conectado al sistema por número de chapa (pasa)
- Probar visualizar en el mapa un taxi no conectado al sistema de ubicación (falla)
- Probar visualizar un conjunto de taxis en un Barrio inexistente (falla)

¡MUCHAS GRACIAS!

- TEST DRIVEN DEVELOPMENT BY EXAMPLE. KENT BECK
- CLEAN CODE. ROBERT C. MARTIN
- DISEÑO AGIL CON TDD - CARLOS BLE JURADO
- ARTÍCULO EMBRACING TEST DRIVEN DEVELOPMENT. BRAD HUETT
- VIDEO: CODELYTV - APRENDE A TRIANGULAR EN TDD PARA PASAR DE ROJO A VERDE

UTN-FRC
INGENIERÍA Y CALIDAD DE SOFTWARE